



Bienvenue dans ce guide ! L'objectif de ce document est de définir précisément notre cadre de travail commun afin de maximiser la vitesse de développement, garantir une sécurité sans faille sur ton code et éliminer tout risque de régression ou de perte de données.

1. Principes Fondamentaux de Notre Collaboration

Travailler en binôme avec **Antigravity** repose sur un principe de **complémentarité stratégique** :



Ton Rôle (Le Visionnaire & Décideur)

- Définir les priorités métier et fonctionnelles.
- Valider l'expérience utilisateur et les visuels.
- Fournir le contexte et arbitrer les choix stratégiques.
- Tester les fonctionnalités en situation réelle.



Mon Rôle (L'Architecte & Exécuteur IA)

- Rédiger, refactoriser et auditer le code TypeScript/React.
- Garantir la conformité avec le Design System & DGI.
- Diagnostiquer et réparer immédiatement les erreurs.
- Vérifier la compilation (`tsc` , `next build`).

2. Vitesse & Sécurité : Les 5 Règles d'Or Zéro-Risque

Pour avancer rapidement sans jamais casser l'existant, nous appliquons cette routine rigoureuse :

PILIER	BONNE PRATIQUE	BÉNÉFICE CLÉ
1. Micro-étapes	Traiter une seule fonctionnalité ou écran à la fois plutôt que 5 chantiers en parallèle.	CONTRÔLE TOTAL Réduction des bugs à 0%.
2. Git Milestones	Effectuer des commits réguliers après chaque fonctionnalité stable et validée.	FILET DE SÉCURITÉ Retour en arrière en 1 clic.
3. Validation Continue	Exécution automatique de <code>npx tsc --noEmit</code> et <code>npm run build</code> avant validation.	ZÉRO RÉGRESSION Détection des erreurs à chaud.
4. Règles Métier	Respect strict des fichiers <code>GEMINI.md</code> et <code>AGENTS.md</code> (Mauritanie, DGI, MRU, Bankily).	INTÉGRITÉ LOCALE Cohérence comptable parfaite.
5. Données Secrètes	Secrets dans <code>.env.local</code> (non commité), données factices pour les tests.	SÉCURITÉ MAX Clés API et Supabase protégées.



Règle de sécurité absolue : Je ne supprime jamais de code ou de fichiers sans avoir une sauvegarde ou une version fonctionnelle de remplacement. Tes modifications sont systématiquement validées en amont.

3. Ce qu'il Nous Faut Installer (Outils & Extensions)

Pour un confort de travail optimal sur VS Code / Antigravity, voici notre boîte à outils recommandée :

Rainbow CSV

Statut : Déjà installé !

Coloration immédiate des colonnes pour inspecter les exports de factures et comptabilité sans tableur externe.

Tailwind CSS IntelliSense

Recommandé

Auto-complétion et aperçu direct des couleurs et classes CSS de notre Design System Facturim.

Pretty TypeScript Errors

Recommandé

Traduit les erreurs TypeScript en explications visuelles claires et limpides.

Supabase VS Code Extension / Studio

Permet d'inspecter visuellement les tables de données PostgreSQL (factures, clients, devis) et tester les requêtes SQL en local.

GitLens ou Git Graph

Visualisation claire de l'historique des commits, des branches et des modifications de code ligne par ligne.

4. Méthodologie de Travail Recommandée (Cycle en 4 Temps)

Chaque session de travail suit ce cycle éprouvé pour une efficacité maximale :

- 1 Cadrage & Intention** : Tu m'exprimes ce que tu souhaites accomplir (ex: « *Ajoutons un filtre par date sur les factures* »).
- 2 Proposition & Plan** : Je t'explique en 2 lignes l'approche technique retenue et les fichiers touchés.
- 3 Exécution de Précision** : Je modifie ou crée les composants selon le Design System (Sky/Cyan, Clean Light, multilingue AR/FR/EN/ZH).
- 4 Vérification & Validation** : On teste visuellement le rendu et je m'assure qu'aucune erreur de compilation n'existe.

5. Le Travail en « Loop » (Mode Autonome & Itératif)



Réponse claire : OUI, je peux travailler en boucle autonome !

Je peux exécuter des séquences complexes d'actions de façon autonome pour te faire gagner un temps précieux.

Voici comment fonctionne le travail en loop avec Antigravity :

- Commande /goal** : Tu me donnes un objectif complexe (ex: « *Développe l'ensemble du module de relance automatique et vérifie avec un test complet* »), et je vais boucler en écrivant le code, exécutant les tests, corrigeant les éventuels bugs et ré-exécutant jusqu'à ce que tout soit parfait.
- Boucle d'auto-correction (Self-Healing Loop)** : Si une commande échoue ou qu'un composant a une erreur de typage, je détecte l'erreur tout seul, j'analyse la cause, je corrige le code et je relance

sans avoir besoin de te déranger.

- **Planification** `/schedule` : Possibilité de programmer des tâches de surveillance ou des vérifications périodiques en arrière-plan.
- **Garde-fous de sécurité** : Même en mode loop, je m'arrête toujours pour te demander validation avant toute opération sensible (suppression de données, migration irréversible).

6. Les Slash Commands Essentielles à Connaître

Dans l'interface de discussion, tu peux utiliser ces raccourcis puissants :

COMMANDE	QUAND L'UTILISER ?	ACTION DÉCLENCHÉE
<code>/goal</code>	Pour les tâches complexes ou longues.	Lance un mode d'exécution autonome jusqu'à réussite totale de l'objectif.
<code>/grill-me</code>	Avant de démarrer un gros module.	Je te pose des questions ciblées pour affiner le besoin avant d'écrire une ligne de code.
<code>/learn</code>	Quand tu souhaites que je retienne une préférence spécifique.	Enregistre une règle personnalisée persistante pour toutes les prochaines sessions.
<code>/schedule</code>	Pour des rappels ou vérifications différées.	Programme un minuteur ou une tâche récurrente en arrière-plan.

7. Checklist Quotidienne de Productivité (Cheat Sheet)

🚀 Au Démarrage d'une Session

- ☐ Vérifier que le serveur local tourne (`npm run dev`).
- ☐ Préciser l'objectif prioritaire du jour.
- ☐ Travailler sur une branche propre ou un commit à jour.

✅ En Fin de Session

- ☐ Vérification globale : `npx tsc --noEmit` .
- ☐ Test visuel rapide sur les écrans modifiés.
- ☐ Commit Git récapitulatif avec message explicite.

8. Résumé de nos Engagements Mutuels



Notre Contrat de Collaboration :

- Zéro perte de temps** : Explications concises, code direct et sans détour.
- Zéro régression** : Tout code touché est testé et validé avant de passer à la suite.
- Qualité Design 100%** : Respect absolu de la charte Facturim (Sky/Slate, Typographie soignée, localisation Mauritanie MRU / 16% TVA).
- Autonomie maîtrisée** : Prêt à enchaîner les tâches en mode loop dès que tu donnes le feu vert !